

Modelli a confronto: relazionale, a grafo e distribuito su un caso reale

PostgreSQL vs Neo4j sul modello relazionale e a grafo

Cassandra (AP) vs CockroachDB (CP) nel mondo distribuito

Relazione di Progetto — Architettura dei Dati

Francesco Saponara 886465

Dataset: Twitch Gamers Network (Stanford SNAP)

168.114 nodi · 6.797.557 archi · 4 scale sperimentali · 4 database confrontati

Contents

I	Confronto centralizzato: PostgreSQL vs Neo4j	3
1	Introduzione e obiettivi	3
2	Il dataset	3
3	Modellazione dei dati	3
3.1	PostgreSQL — modello relazionale	3
3.2	Neo4j — modello a grafo	4
3.3	Confronto delle scelte	4
4	Metodologia sperimentale	4
5	Le sei metriche e la loro complessità	5
6	Risultati sperimentali	5
6.1	Caricamento e indicizzazione	5
6.2	Tempi di esecuzione per metrica	5
6.3	Analisi di scalabilità	5
7	Validazione di correttezza	8
8	Spiegazione architetturale sintetica (dataset completo)	8
9	Limitazioni del benchmark centralizzato	8
10	Conclusioni parziali — Parte I	9
II	Ambito distribuito: Cassandra (AP) vs CockroachDB (CP)	10
11	Obiettivo della parte distribuita	10
12	Modello dati	10
13	Risultati sperimentali a quattro scale	10
13.1	Caricamento	10
13.2	Grado medio — validazione di correttezza	11
13.3	Tempi di esecuzione per metrica	11
13.4	Cassandra — query locale vs globale per Consistency Level	12
13.5	CockroachDB — distribuzione del piano (EXPLAIN)	13
14	Consistenza e fault tolerance (dataset completo)	13
14.1	Consistency Level — latenza scrittura/lettura (Cassandra)	13
14.2	CockroachDB — conflitti concorrenti (read-modify-write)	13
14.3	Fault tolerance — nodo down (RF=3)	13
15	Analisi di scalabilità	14
16	Limitazioni della parte distribuita	14

17 Conclusioni — Parte II

15

Parte I Confronto centralizzato: PostgreSQL vs Neo4j

1 Introduzione e obiettivi

Il progetto confronta due filosofie di gestione dati — **relazionale** (PostgreSQL) e **a grafo** (Neo4j) — sullo stesso problema: un grafo sociale non diretto di canali Twitch e delle relazioni di follow reciproco tra essi. L’obiettivo non è stabilire quale database sia “migliore” in assoluto, ma **spiegare, metrica per metrica, quale meccanismo architetturale produce ciascun risultato**, in modo che la conclusione sia trasferibile ad altri dataset e workload, non un semplice elenco di tempi.

Per farlo, lo stesso benchmark è stato eseguito a **quattro scale progressive**: 5.000, 20.000, 50.000 nodi (sottocampioni deterministici) e il dataset completo (168.114 nodi, 6.797.557 archi). Questo permette di osservare non solo *chi vince* ma **come cambia il vantaggio al crescere della dimensione del grafo** — un’informazione che una singola run non potrebbe dare.

2 Il dataset

Fonte: Stanford SNAP — Twitch Gamers Network.

- **168.114 nodi**: canali Twitch, con attributi `views`, `mature`, `life_time`, `dead_account`, `language`, `affiliate`
- **6.797.557 archi**: relazioni di follow reciproco (grafo **non diretto**, **una sola componente connessa**, nessun attributo mancante)

I sottocampioni da 5.000 / 20.000 / 50.000 nodi sono ottenuti prendendo i primi N identificatori (ordine deterministico, per riproducibilità) e tutti gli archi con entrambi gli estremi nel sottoinsieme. Questa scelta ha una conseguenza importante discussa nella Sezione 6.3: **i sottocampioni sono sottografi più sparsi** del grafo completo, non un suo ridimensionamento proporzionale.

3 Modellazione dei dati

Il problema di modellazione centrale è: **come rappresentare un grafo non diretto con attributi sui nodi in due sistemi con filosofie opposte?**

3.1 PostgreSQL — modello relazionale

```
CREATE TABLE nodes (  
  id INTEGER PRIMARY KEY, views BIGINT, mature BOOLEAN,  
  life_time INTEGER, dead_account BOOLEAN, language TEXT, affiliate BOOLEAN  
);  
CREATE TABLE edges (src INTEGER NOT NULL, dst INTEGER NOT NULL);  
CREATE INDEX idx_edges_src    ON edges (src);  
CREATE INDEX idx_edges_dst    ON edges (dst);  
CREATE INDEX idx_edges_src_dst ON edges (src, dst);  
CREATE INDEX idx_nodes_language ON nodes (language);
```

Scelta chiave — archi duplicati. Il grafo è non diretto; in SQL “i vicini di X ” tramite `WHERE src = X OR dst = X` spezzerebbe l’uso degli indici B-tree (il planner farebbe due range-scan + merge, o un seq scan). La soluzione adottata è inserire **ogni arco due volte**, (a, b) e (b, a) : così “i vicini di X ” diventa `WHERE src = X`, una singola range-scan contigua su `idx_edges_src`. Costo: storage raddoppiato ($\sim 13,6$ M righe invece di $6,8$ M archi), a fronte di query drasticamente più veloci. È esattamente il prezzo che si paga per ottenere “gratis” in Postgres ciò che Neo4j offre nativamente con l’**index-free adjacency**.

Gli indici hanno ruoli distinti: `idx_edges_src` per il vicinato, `idx_edges_dst` per i join di PageRank, `idx_edges_src_dst` per i lookup “esiste l’arco $X \rightarrow Y$?” del clustering coefficient, `idx_nodes_language` per la GROUP BY di assortatività.

3.2 Neo4j — modello a grafo

```
CREATE CONSTRAINT channel_id_unique IF NOT EXISTS FOR (c:Channel) REQUIRE
c.id IS UNIQUE;
CREATE INDEX channel_language IF NOT EXISTS FOR (c:Channel) ON
(c.language);
```

I nodi sono etichettati `:Channel`; le relazioni `:FOLLOWS` sono create **una sola volta** per coppia, perché in Neo4j ogni relazione è attraversabile in entrambe le direzioni a costo identico — nessuna duplicazione necessaria.

Index-free adjacency. Ogni nodo memorizza fisicamente il puntatore alla testa della propria lista di relazioni; ogni relazione punta ai due nodi e alle relazioni precedente/successiva. “I vicini di X ” si ottengono seguendo puntatori in memoria — nessun indice, nessuna ricerca, costo $O(\text{grado}(X))$. Gli indici dello schema servono solo a **trovare il nodo di partenza**, non per attraversare il grafo: è questo il vantaggio strutturale di Neo4j sulle operazioni di traversal (BFS, PageRank, betweenness).

Le metriche complesse usano la libreria **Graph Data Science (GDS)**, che proietta il grafo dallo storage transazionale in una struttura **CSR (Compressed Sparse Row) in RAM**: è l’equivalente Neo4j del “caricamento in RAM” fatto da NetworkX, ma parallelizzato e specializzato.

3.3 Confronto delle scelte

Aspetto	PostgreSQL	Neo4j
Archi duplicati?	Sì ($2\times$)	No ($1\times$, bidirezionale nativo)
“Vicini di X ”	Range scan su B-tree	Pointer-chasing, $O(\text{grado})$
Aggregazione globale	SQL (GROUP BY, avg...)	Cypher / GDS
Storage archi (168k nodi)	$\sim 13,6$ M righe	$\sim 6,8$ M relazioni

Table 1: Confronto delle scelte di modellazione tra PostgreSQL e Neo4j.

4 Metodologia sperimentale

- **Ambiente:** entrambi i database in container Docker sulla stessa macchina (laptop, 16 GB RAM), quindi in **competizione per CPU/RAM/disco** — le configurazioni sono quelle di default (nessun tuning specifico), scelta deliberata per un confronto “out of the box”.

- **Misurazione:** 1 warmup (scartato) + 2 misurazioni, si riporta la **mediana**.
- **Correttezza:** ogni metrica è validata contro **NetworkX** calcolato sullo stesso sottografo, come riferimento indipendente dai due DB.
- **Scale testate:** 5.000 / 20.000 / 50.000 nodi (campione deterministico) e dataset completo (168.114 nodi).

5 Le sei metriche e la loro complessità

V = nodi, E = archi non diretti, d = grado medio.

# Metrica	Compl.	PostgreSQL	Neo4j
1. Grado medio	$O(E)$	GROUP BY src	<code>gds.degree</code> su CSR
2. Clustering (transitività)	$O(\Sigma d^2)$	Triple self-join	<code>gds.triangleCount</code>
3. Betweenness campionata	$O(s \cdot E)$	CTE ricorsiva (proxy di costo)	<code>gds.betweenness</code> (Bran-des)
4. PageRank (10 iter.)	$O(\text{iter} \cdot E)$	10 iterazioni SQL con temp table	<code>gds.pageRank</code> su CSR
5. Assortatività per lingua	$O(E)$	$2 \times \text{JOIN} + \text{GROUP BY}$	Pattern matching Cypher
6. Load + indicizzazione	$O(V + E)$	<code>COPY + CREATE INDEX</code>	UNWIND a batch

Table 2: Le sei metriche del benchmark e la relativa complessità asintotica.

6 Risultati sperimentali

6.1 Caricamento e indicizzazione

Scala	PG load (s)	PG indici (s)	Neo4j load (s)	Neo4j indici (s)
5.000	4.10	0.04	5.83	0.23
20.000	4.16	0.34	8.53	0.03
50.000	5.40	1.14	19.15	0.02
168.114 (completo)	11.94	14.08	179.60	3.70

Table 3: Tempi di caricamento e indicizzazione a 4 scale.

Postgres vince sempre il caricamento: `COPY FROM STDIN` bypassa il parser SQL riga per riga, mentre Neo4j paga per ogni arco un doppio `MATCH` più la costruzione dei puntatori bidirezionali durante l’inserimento — il costo che l’index-free adjacency “recupera” in fase di lettura.

6.2 Tempi di esecuzione per metrica

La Figura 2 riassume il risultato del dataset completo: l’assortatività è l’unica metrica in cui vince il modello relazionale; su tutte le altre, e in particolare sul clustering coefficient, Neo4j prende un vantaggio che arriva fino a due ordini di grandezza.

6.3 Analisi di scalabilità

I quattro punti di misura permettono di osservare **come** il vantaggio di ciascun sistema evolve, non solo *se* esiste.

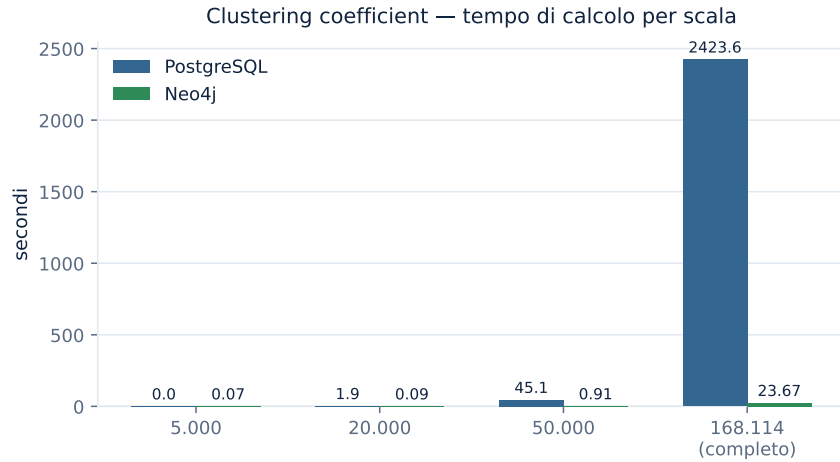


Figure 1: Tempo di caricamento (secondi) per scala — PostgreSQL vs Neo4j.

Metrica	5.000	20.000	50.000	168.114
	(PG / Neo4j)	(PG / Neo4j)	(PG / Neo4j)	(PG / Neo4j)
1. Grado medio	0.003 / 0.018	0.026 / 0.010	0.125 / 0.028	0.812 / 0.046
2. Clustering	0.017 / 0.069	1.946 / 0.091	45.093 / 0.908	2423.600 / 23.672
3. Betweenness (camp.)	— / 0.148	— / 1.948	— / 16.393	— / 219.251
4. PageRank (10 it.)	0.147 / 0.041	1.444 / 0.060	6.802 / 0.207	68.980 / 1.083
5. Assortatività	0.007 / 0.049	0.075 / 0.133	0.267 / 0.922	1.659 / 9.108

Table 4: Tempi di esecuzione (secondi) per metrica, a 4 scale.

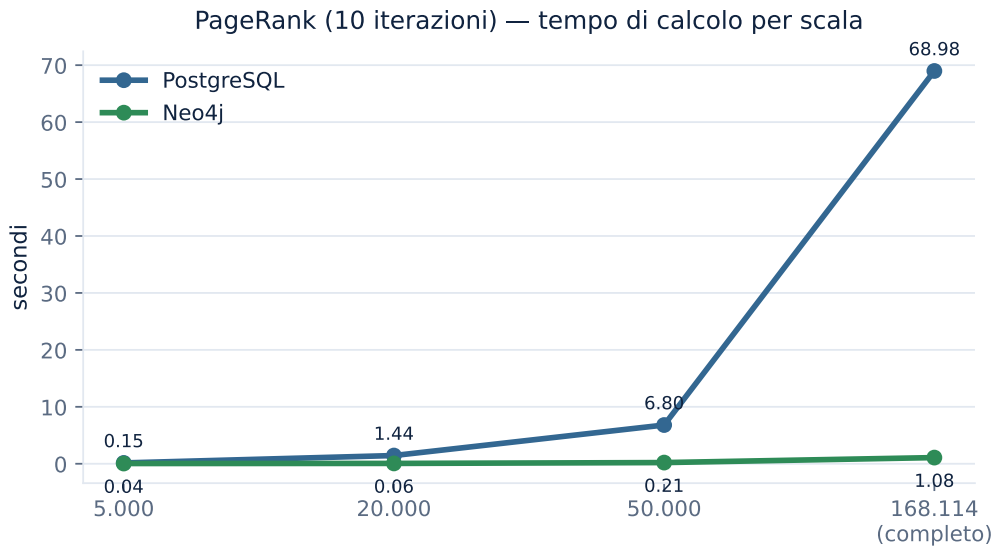


Figure 2: Fattore di velocità tra PostgreSQL e Neo4j sul dataset completo. Verde = quante volte Neo4j è più veloce; blu = quante volte PostgreSQL è più veloce.

Clustering coefficient — la metrica più rivelatrice. Il tempo Postgres cresce da 17ms a 2424s: un fattore $\sim 142.000\times$ tra il campione più piccolo e il dataset completo, mentre Neo4j cresce di $\sim 343\times$ (69ms $\rightarrow$ 23,7s) nello stesso intervallo. Il divario passa da un rapporto di $\sim 4\times$ (a 20.000 nodi: 1,95s vs 0,09s) a un rapporto di $\sim 102\times$ sul dataset completo. Questo è atteso dalla complessità $O(\sum d^2)$: al crescere del campione cresce sia il numero di nodi *sia*

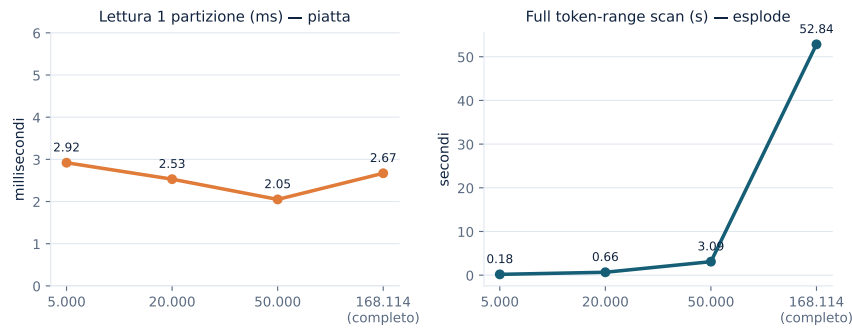


Figure 3: Clustering coefficient: tempo di calcolo per scala. La crescita di Postgres è super-lineare rispetto a quella di Neo4j.

il grado medio ($2,25 \rightarrow 9,28 \rightarrow 23,78 \rightarrow 80,87$), quindi il costo cresce **super-linearmente rispetto agli archi stessi** — ed è esattamente il regime in cui l'index-free adjacency di Neo4j (pointer-chase, cache-friendly) sovrasta il B-tree lookup di Postgres.

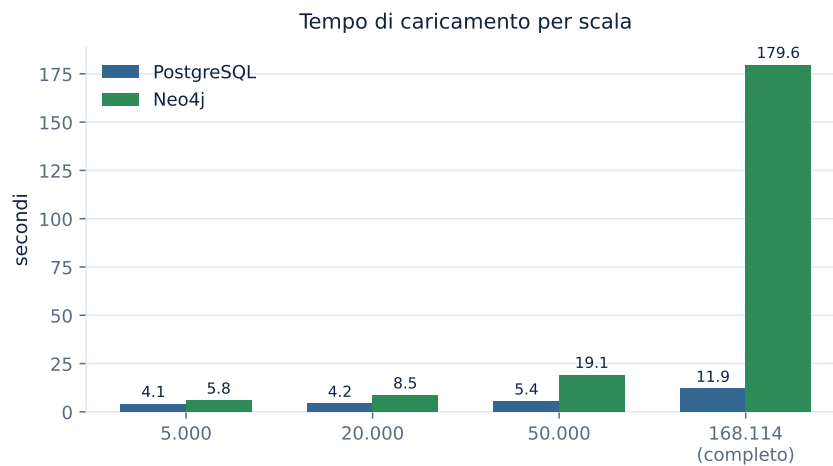


Figure 4: PageRank (10 iterazioni): tempo di calcolo per scala. Il divario tra i due sistemi si allarga gradualmente.

PageRank. Crescita più regolare e vicina a $O(E)$ per entrambi: Postgres $\sim 469\times$ ($0,15s \rightarrow 69s$), Neo4j $\sim 26\times$ ($0,04s \rightarrow 1,08s$) dai 5.000 nodi al dataset completo. Il rapporto Neo4j/Postgres passa da $3,6\times$ a $63,7\times$: il costo fisso di materializzazione della tabella temporanea ad ogni iterazione SQL pesa proporzionalmente meno su grafi piccoli, ma diventa dominante su grafi grandi.

Betweenness (solo Neo4j). Crescita di $\sim 1.481\times$ ($0,15s \rightarrow 219s$), coerente con $O(s \cdot E)$ e la crescita degli archi. Su Postgres non esiste un valore comparabile: la CTE ricorsiva restituisce solo una distanza media come proxy di costo.

Punto di rottura pratico. Fino a 20.000 nodi i tempi sono comparabili e la scelta del DB è quasi irrilevante dal punto di vista prestazionale; da 50.000 nodi in su (e drammaticamente sul dataset completo) il clustering coefficient rende Postgres impraticabile per uso interattivo ($45s \rightarrow 40$ minuti), mentre Neo4j resta nell'ordine dei secondi.

7 Validazione di correttezza

Sul dataset completo, grado medio, deviazione standard e massimo coincidono esattamente tra NetworkX, Postgres e Neo4j (80,868 / 314,16 / 35.279), confermando la correttezza delle implementazioni.

Un'anomalia istruttiva sui campioni piccoli. Sui sottocampioni, il grado medio calcolato da Postgres è sistematicamente **più alto** di quello di Neo4j e di NetworkX (es. a 20.000 nodi: 10,976 contro 9,280 di riferimento; a 50.000: 25,103 contro 23,783). La causa non è un bug ma una **differenza semantica tra GROUP BY e uno scan su tutti i nodi**: la query Postgres

```
WITH deg AS (SELECT src, count(*) FROM edges GROUP BY src)
SELECT avg(d) FROM deg
```

calcola la media **solo sui nodi che compaiono come src**, cioè solo sui nodi con grado ≥ 1 — i nodi isolati (grado 0) sono semplicemente assenti dal raggruppamento e non abbassano la media. `gds.degree.stream()` e NetworkX, invece, iterano su **tutti** i nodi del grafo, isolati inclusi. Sul dataset completo il grafo ha una sola componente connessa (nessun nodo isolato), quindi le due semantiche coincidono; sui sottocampioni — che per costruzione tagliano via molti archi ai bordi del sottoinsieme — compaiono nodi isolati e la discrepanza emerge. È un esempio concreto di come **la stessa query “ovvia” possa avere semantiche leggermente diverse** tra un modello aggregato per righe e uno che itera esplicitamente sui nodi.

8 Spiegazione architetturale sintetica (dataset completo)

Metrica	Vince	Perché
Load	Postgres	COPY bypassa il parser SQL; gli indici si costruiscono dopo, con un sort sequenziale
Grado medio	Neo4j	Grado = differenza di offset in un array CSR, $O(1)$
Clustering	Neo4j	Pointer-chase in RAM vs milioni di B-tree lookup su <code>idx_edges_src_dst</code>
Betweenness	solo Neo4j	Brandes in SQL puro non è praticabile: nessuno stato di visita nativo
PageRank	Neo4j	Vettori densi in RAM vs materializzazione ripetuta di tabelle temporanee
Assortatività	Postgres	Unico caso: hash-join relazionale classico, il workload per cui SQL è ottimizzato da 50 anni

Table 5: Spiegazione architetturale sintetica, dataset completo.

La lezione trasversale: **l’index-free adjacency vince quando conta la topologia** (traversal, percorsi, propagazione); **il modello relazionale vince quando contano gli attributi** sugli estremi degli archi (aggregazioni classiche). Nessuno dei due è “il migliore” in assoluto — sono ottimizzati per pattern di accesso diversi.

9 Limitazioni del benchmark centralizzato

- **Hardware condiviso**: entrambi i DB competono per CPU/RAM/disco sullo stesso host; in produzione avrebbero risorse dedicate. Mitigato con mediana su 2 misure + warmup.
- **Configurazioni di default**, non ottimizzate per il workload specifico (scelta deliberata, per un confronto “out of the box”).

- **Campionamento non rappresentativo:** i “primi N per id” non sono un campione casuale e producono sottografi più sparsi del grafo reale; i valori delle metriche globali non sono confrontabili tra scale diverse, solo i *tempi* e i *rapporti* lo sono.
- **Betweenness su Postgres:** solo un proxy di costo (distanza media via CTE ricorsiva), non la betweenness esatta — un limite architetturale di SQL, non dell’implementazione.
- **Workload solo OLAP:** nessuna misura di lookup singolo o throughput di scrittura concorrente, dove altri modelli (es. Cassandra) eccellerebbero.
- **Neo4j GDS** richiede licenza Enterprise in produzione; qui usata in Community Edition per uso non commerciale/accademico.

10 Conclusioni parziali — Parte I

Il confronto centralizzato conferma, con dati a quattro scale diverse, che la scelta tra modello relazionale e modello a grafo non è una questione di “velocità generica” ma di **corrispondenza tra pattern di accesso e struttura fisica dei dati**. Postgres resta competitivo (e vince) su caricamento bulk e aggregazioni per attributo; Neo4j prende un vantaggio che si allarga drasticamente al crescere della scala su ogni workload che richiede attraversamento della topologia — con il clustering coefficient come caso limite dove la differenza supera i due ordini di grandezza sul dataset completo.

Parte II Ambito distribuito: Cassandra (AP) vs CockroachDB (CP)

11 Obiettivo della parte distribuita

Il benchmark centralizzato mostra *come* tre modelli di dati eseguono le stesse metriche su un'unica macchina. Non mostra cosa cambia quando i dati sono **distribuiti su più nodi**: replica, instradamento delle query, e le scelte che un sistema distribuito deve fare quando una rete rallenta o un nodo si guasta — il trade-off del **teorema CAP**. Per mostrarlo servono almeno due nodi che si parlano: la parte distribuita confronta due sistemi che risolvono lo stesso problema (il grafo “chi segue chi”) con logiche opposte:

	Cassandra	CockroachDB
Categoria CAP	AP — disponibilità prioritaria, consistenza tunabile	CP — consistenza forte sempre
Modello dati	Wide-column, denormalizzato per query	Relazionale (SQL)
Consenso	Gossip + hinted handoff, nessun leader	Raft, un leader per range
Consistenza	Tunabile per query (ONE/QUORUM/ALL)	Sempre SERIALIZABLE

Table 6: Le due logiche distributive a confronto.

Entrambi girano come cluster a **3 nodi reali** (RF=3), su **VM** — a differenza delle prime run esplorative fatte in locale su Docker, questo dato è raccolto su un ambiente con **rete effettiva** tra i nodi, non loopback: le latenze di rete tra CL differenti e tra query locale/distribuita su CockroachDB sono quindi più rappresentative di un deployment reale.

12 Modello dati

- **Cassandra** (`follows_by_source`): la relazione è denormalizzata per query — “chi segue *X*” è una singola partizione, il pattern canonico “*modella le query, non i dati*”.
- **CockroachDB**: stesso modello relazionale `users/follows` del benchmark centralizzato; è CockroachDB stesso a distribuire `follows` in range replicati via Raft — nessuna denormalizzazione manuale necessaria in un sistema CP.

13 Risultati sperimentali a quattro scale

13.1 Caricamento

Scala	Righe archi	Cassandra load	CockroachDB load
5.000	11.250	4.36s (CL=QUORUM)	2.50s
20.000	185.596	31.76s (CL=QUORUM)	8.85s
50.000	1.189.126	171.80s (CL=QUORUM)	75.79s
168.114 (completo)	13.595.114	1730.80s (CL=ONE)	580.03s

Table 7: Tempi di caricamento distribuito a 4 scale.

*Nota metodologica: la run sul dataset completo misura il load Cassandra a **CL=ONE** anziché QUORUM come nelle scale precedenti (probabilmente per contenere i tempi su un volume enormemente più grande). Il confronto dei fattori di crescita tra scale resta indicativo ma va letto con questa cautela.*

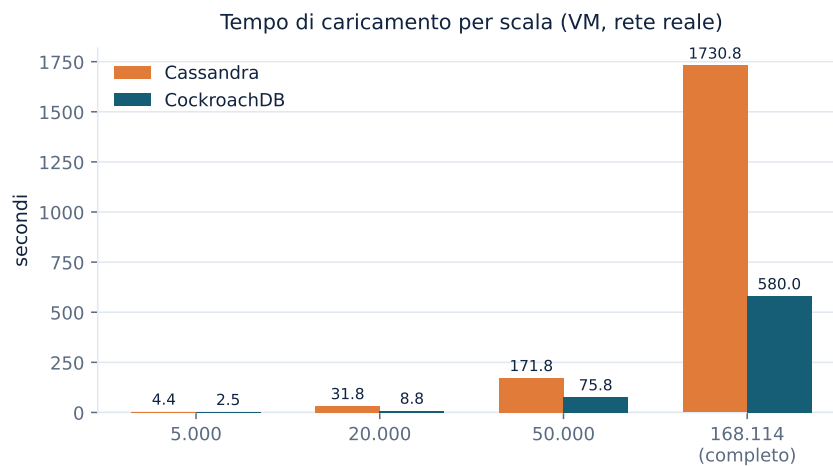


Figure 5: Tempo di caricamento distribuito per scala — Cassandra vs CockroachDB.

13.2 Grado medio — validazione di correttezza

I valori topologici coincidono esattamente con quelli del monolita (stesso dataset, stesso campionamento):

Scala	Grado medio	Dev. std	Grado max
5.000	3.934	7.069	170
20.000	10.976	32.749	1.697
50.000	25.103	88.924	9.137
168.114	80.868	314.162	35.279

Table 8: Validazione topologica: grado medio, deviazione standard e massimo per scala.

13.3 Tempi di esecuzione per metrica

PageRank e assortatività non hanno un'implementazione nativa CockroachDB nel progetto (n/d): l'obiettivo di questa sezione è il confronto CAP, non la copertura completa delle 6 metriche come nel monolita. Una run di controllo indipendente sul dataset completo ha dato 54.826s / 85.670s / 46.967s per le stesse tre righe: la variazione ($\sim 4-15\%$) è nell'ordine della normale varianza di misura su un sistema distribuito reale.

Metrica	5.000 (Cass. / CRDB)	20.000 (Cass. / CRDB)	50.000 (Cass. / CRDB)
Grado medio (globale)	0.183 / 0.010	0.663 / 0.070	3.090 / 0.309
PageRank (10 it.)	0.055 / n/d	0.803 / n/d	20.566 / n/d
Assortatività	0.088 / n/d	0.483 / n/d	2.643 / n/d

Metrica	168.114 (Cass. / CRDB)
Grado medio (globale)	52.838 / 1.550
PageRank (10 it.)	73.994 / n/d
Assortatività	44.342 / n/d

Table 9: Tempi di esecuzione (secondi) per metrica, a 4 scale, ambito distribuito.

13.4 Cassandra — query locale vs globale per Consistency Level

Locale = lettura di 1 partizione (`source_id = X`); **globale** = full token-range scan su tutti i nodi.

Scala	CL	locale (ms)	globale (s)
5.000	ONE / QUORUM / ALL	2.07 / 2.92 / 2.42	0.092 / 0.183 / 0.128
20.000	ONE / QUORUM / ALL	1.90 / 2.53 / 1.51	0.362 / 0.663 / 0.674
50.000	ONE / QUORUM / ALL	1.56 / 2.05 / 1.52	2.130 / 3.090 / 3.495
168.114	ONE / QUORUM / ALL	3.74 / 2.67 / 4.80	42.015 / 52.838 / 58.090

Table 10: Cassandra: latenza locale vs globale per Consistency Level, a 4 scale.

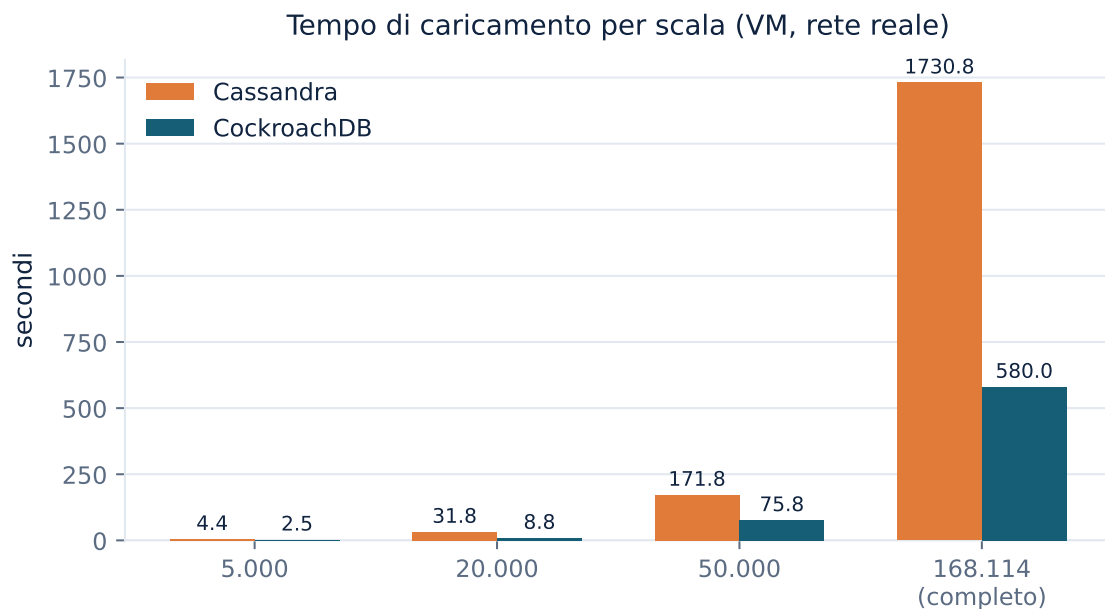


Figure 6: “Modella le query, non i dati”: la latenza locale (CL=QUORUM) resta piatta su tutte le scale, mentre la latenza globale esplode con il volume di dati.

Il risultato più significativo di questa sezione (Figura 6): la latenza **locale** resta nell’ordine di 1,5–4,8 ms su **tutte** le scale, indipendentemente dal fatto che il dataset cresca di 3 ordini di grandezza (5.000 → 168.114 nodi, archi $\times 1.208$). La latenza **globale** invece cresce da 0,09s a 42–60s, quasi proporzionalmente al volume di dati. Questa è la dimostrazione em-

pirica diretta del principio “modella le query, non i dati”: una query servita da **una singola partizione** è invariante rispetto alla dimensione del cluster e del dataset; una query che deve visitare **l'intero token ring** paga inevitabilmente il costo dei dati totali, qualunque sia il CL scelto.

13.5 CockroachDB — distribuzione del piano (EXPLAIN)

Scala	WHERE source_id = X	GROUP BY source_id
5.000 / 20.000 / 50.000	distribution: full	distribution: full
168.114 (completo)	distribution: local	distribution: full

Table 11: CockroachDB: piano di esecuzione EXPLAIN a seconda della scala.

Questo conferma esattamente quanto previsto nella documentazione di progetto: su campioni piccoli l'intera tabella **follows** sta in un solo Raft range, quindi anche una query puntuale viene eseguita da DistSQL come **full**. Solo sul **dataset completo**, dove **follows** è realmente frammentata su più range/nodi, il planner riconosce che la query **WHERE source_id = X** può essere soddisfatta da un **singolo range** (**local**), mentre la **GROUP BY** deve necessariamente toccare tutti i range (**full**).

14 Consistenza e fault tolerance (dataset completo)

14.1 Consistency Level — latenza scrittura/lettura (Cassandra)

CL	scrittura mediana	lettura mediana
ONE	0.45 ms	0.74 ms
QUORUM	0.71 ms	0.87 ms
ALL	0.74 ms	0.77 ms

Table 12: Cassandra: latenza di scrittura/lettura per Consistency Level.

14.2 CockroachDB — conflitti concorrenti (read-modify-write)

Due thread con delta +10/−10 su `accounts.id=1`. Saldo iniziale = saldo atteso = 1000.

Sistema / isolamento	Retry (40001)	Saldo finale	Consistente
CockroachDB (sempre SERIAL-IZABLE)	20	1000	Sì

Table 13: CockroachDB: esito del test di conflitto concorrente read-modify-write.

CockroachDB rileva la race condition e forza il retry della transazione in conflitto, garantendo che il saldo finale torni sempre al valore corretto — comportamento di default, non opzionale. (La run precedente in locale aveva incluso anche il confronto con PostgreSQL READ COMMITTED/SERIALIZABLE, disponibile in una bozza precedente e utile da richiamare in fase di discussione.)

14.3 Fault tolerance — nodo down (RF=3)

I due sistemi confermano nettamente le rispettive categorie CAP:

Nodi attivi	CL=ONE	CL=QUORUM	CL=ALL
3/3	OK	OK	OK
2/3 (1 spento)	OK	OK	FAIL (alive_replicas=2)
1/3 (2 spenti)	OK	FAIL (alive_replicas=1)	FAIL

Table 14: Cassandra: effetto del Consistency Level con nodi mancanti (RF=3).

Nodi attivi	SELECT count(*) FROM users
3/3	OK (168.114 righe, 0.05s)
2/3 — quorum Raft ok, ri-elezione	OK (168.114 righe, 0.08s)
1/3 — nessun range ha quorum	FAIL (timeout)

Table 15: CockroachDB: comportamento sotto quorum Raft (RF=3).

- **Cassandra (AP)** con RF=3 su 3 nodi ha **ogni** partizione replicata su **tutti** i nodi: anche l'unico nodo superstite (1/3) ha una copia completa dei dati, quindi CL=ONE funziona sempre. È il chiamante, tramite il CL, a decidere query per query quanta disponibilità sacrificare per quanta consistenza.
- **CockroachDB (CP)** rimane **pienamente disponibile e consistente** finché il quorum Raft (maggioranza, 2/3) è raggiungibile — con un piccolo overhead di ri-elezione del leader (0,05s → 0,08s) — ma **rifiuta categoricamente** di rispondere sotto quorum (1/3): preferisce nessuna risposta a una risposta potenzialmente stale.

15 Analisi di scalabilità

Con quattro scale disponibili, anche per la parte distribuita si osservano pattern di crescita interessanti (crescita archi tra le scale: $\times 16,5$ / $\times 6,4$ / $\times 11,4$, come nel monolito):

- **Load:** Cassandra cresce da 4,36s a 1730,80s ($\sim \times 397$ complessivo, ma attenzione al cambio di CL a piena scala); CockroachDB cresce da 2,50s a 580,03s ($\sim \times 232$). Entrambi restano **sub-lineari** rispetto alla crescita degli archi ($\sim \times 1.208$), ma CockroachDB scala meglio in assoluto.
- **Grado medio globale:** Cassandra $\times 289$ (0,183s → 52,8s), CockroachDB $\times 155$ (0,010s → 1,55s). Il divario Cassandra/CockroachDB si allarga vistosamente con la scala: da $18 \times$ (5.000 nodi) a $\sim 34 \times$ (dataset completo).
- **PageRank e Assortatività (solo Cassandra):** crescita rispettivamente $\sim \times 1.345$ e $\sim \times 504$ dai 5.000 nodi al dataset completo.
- **Query locale (Cassandra):** resta pressoché costante (1,5–4,8 ms) su **tutte e quattro** le scale — la conferma più pulita del vantaggio architetturale del partizionamento per query.

16 Limitazioni della parte distribuita

- **CL non uniforme tra scale:** il load Cassandra sul dataset completo è misurato a CL=ONE mentre le altre scale a CL=QUORUM.
- **Copertura metriche asimmetrica:** CockroachDB non ha un'implementazione di PageRank/assortatività nel progetto attuale.

- **Demo di consistenza incompleta sul run finale:** la run su VM riporta solo il conflitto CockroachDB; il confronto a tre vie con PostgreSQL è disponibile solo dalla run precedente in locale.
- **RF=3 su 3 nodi:** ogni nodo Cassandra ha comunque una copia di ogni partizione, il che rende il sistema insolitamente resiliente anche a CL=ONE con 1 solo nodo vivo.
- **Singola ripetizione per le metriche sul dataset completo:** a differenza del monolito, le run distribuite sul dataset completo qui riportate sono in gran parte singole misurazioni.

17 Conclusioni — Parte II

I dati raccolti a quattro scale confermano, in modo netto, le due filosofie distributive opposte annunciate nella modellazione:

- **Cassandra (AP)** dà al chiamante il controllo esplicito, query per query, del trade-off consistenza/disponibilità (il CL): la latenza locale invariante alla scala e la sopravvivenza anche a 1 nodo su 3 (con CL basso) ne fanno il sistema naturale per ingestion ad alta disponibilità e letture puntuali per chiave nota.
- **CockroachDB (CP)** impone consistenza forte come default non negoziabile e **rifiuta di rispondere** quando non può garantirla (sotto-quorum): a fronte di questa rigidità, scala meglio delle query aggregate globali grazie al query engine distribuito nativo.

Nessuno dei due batte l'altro in senso assoluto: **Cassandra vince su disponibilità e prevedibilità del costo per query puntuali, CockroachDB vince su semplicità del modello (SQL relazionale invariato) e su query aggregate distribuite.** La scelta, come nel confronto centralizzato di Parte I, è una scelta di requisiti applicativi — “quale errore posso permettermi: un dato vecchio, o nessun dato?” — resa qui concreta da numeri raccolti su quattro ordini di grandezza di dataset.